
تمام فایل های تغییر یافته و عکس خروجی ضمیمه شده است .

(1)

برای این قسمت برای اینکه زمان بندی lottery رو پیاده کنم باید هر پردازش چند تا بلیط داشته باشه بنابراین خط `int tickets;` رو مطابق زیر اضافه کردم تا توی قرعه کشی CPU سهم هر پردازش مشخص بشه .

```
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;          // Process state
    void *chan;                    // If non-zero, sleeping on chan
    int killed;                    // If non-zero, have been killed
    int xstate;                    // Exit status to be returned to parent's wait
    int pid;                       // Process ID
    int tickets;

    // wait_lock must be held when using this:
    struct proc *parent;           // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;                 // Virtual address of kernel stack
    uint64 sz;                     // Size of process memory (bytes)
    pagetable_t pagetable;         // User page table
    struct trapframe *trapframe;   // data page for trampoline.S
    struct context context;        // swtch() here to run process
    struct file *ofile[NOFILE];   // Open files
```

```

    struct inode *cwd;           // Current directory
    char name[16];               // Process name (debugging)
};

```

(2)

برای مقدار دهی اولیه به tickets مطابق کد زیر به تابع alloproc در proc.c کد زیر رو اضافه کردم :

```
p->tickets = DEFAULT_TICKETS;
```

دلیل اینکار هم اینه که وقتی یک پردازش جدید درست میشه یک مقدار پیش فرض بهش بدم تا بتونه تو قرعه کشی شرکت کنه . چون اگه یک مقدار نداشته باشه اون وقت دیگه بلیطی نداره یا مقدار بلیطش نامعتبر هست در نتیجه در اینجا تو schedule به مشکل میخوریم پس باید حتما یک مقدار دیفالت بهش بدیم . پس در کل قسمت هایی که در فایل proc.c تغییر دادم به صورت زیر هست :

```

static struct proc*
alloproc(void)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == UNUSED) {
            goto found;
        } else {
            release(&p->lock);
        }
    }
    return 0;
}

```

found:

```

p->pid = allocpid();
p->state = USED;

```

```

p->tickets = DEFAULT_TICKETS;
if((p->trapframe = (struct trapframe *)kalloc()) == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}
p->pagetable = proc_pagetable(p);
if(p->pagetable == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}
memset(&p->context, 0, sizeof(p->context));
p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;

return p;
}

static void
freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    if(p->pagetable)
        proc_freepagetable(p->pagetable, p->sz);
    p->pagetable = 0;
    p->sz = 0;
    p->pid = 0;
    p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->killed = 0;
    p->xstate = 0;
    p->state = UNUSED;
}

```

```

pagetable_t
proc_pagetable(struct proc *p)
{
    pagetable_t pagetable;
    pagetable = uvmcreate();
    if(pagetable == 0)
        return 0;
    if(mappages(pagetable, TRAMPOLINE, PGSIZE,
                (uint64)trampoline, PTE_R | PTE_X) < 0){
        uvmfree(pagetable, 0);
        return 0;
    }
    , TRAPFRAME, PGSIZE,
        (uint64)(p->trapframe), PTE_R | PTE_W) < 0){
        uvmunmap(pagetable, TRAMPOLINE, 1, 0);
        uvmfree(pagetable, 0);
        return 0;
    }

    return pagetable;
}

```

حالا در بخش بعدی باید fork رو تغییر میدادم تا پردازش فرزند تعداد بلیط هاش رو از پدرش به ارث ببره علتش هم سر اینه که در os فرزند باید حتما ویژگی های پدرش رو به ارث ببره تا اگه تیکت های پدر زیادتیر بود این فرزند نیز این بلیط هارو به ارث بگیره پس خط زیر رو اضافه کردم:

```
np->tickets = p->tickets;
```

و در نهایت تابع kfork و reparent در فایل proc.c به صورت زیر میشه:

```

int
kfork(void)
{
    int i, pid;
    struct proc *np;

```

```

struct proc *p = myproc();

// Allocate process.
if((np = allocproc()) == 0){
    return -1;
}

// Copy user memory from parent to child.
if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
    freeproc(np);
    release(&np->lock);
    return -1;
}
np->sz = p->sz;

// copy saved user registers.
*(np->trapframe) = *(p->trapframe);

// Cause fork to return 0 in the child.
np->trapframe->a0 = 0;

// increment reference counts on open file descriptors.
for(i = 0; i < NOFILE; i++){
    if(p->ofile[i])
        np->ofile[i] = filedup(p->ofile[i]);
}
np->cwd = idup(p->cwd);

safestrcpy(np->name, p->name, sizeof(p->name));

np->tickets = p->tickets;

pid = np->pid;

release(&np->lock);

acquire(&wait_lock);

```

```

np->parent = p;
release(&wait_lock);

acquire(&np->lock);
np->state = RUNNABLE;
release(&np->lock);

return pid;
}

// Pass p's abandoned children to init.
// Caller must hold wait_lock.
void
reparent(struct proc *p)
{
    struct proc *pp;

    for(pp = proc; pp < &proc[NPROC]; pp++){
        if(pp->parent == p){
            pp->parent = initproc;
            wakeup(initproc);
        }
    }
}

```

(3)

در این قسمت باید کدی مینوشتیم که کل بلیط های تمام پردازش های که وضعیتشون runnable هست رو محاسبه کنه دلیل اینکه اینکار رو انجام میدیم این هست که باید بدونیم بازه قرعه کشی یعنی از 0 تا total_tickets-1 چه مقداری داره . در مرحله بعدی مطابق صورت سوال از تابع rand_int() استفاده میکنم تا یک عدد تصادفی بسازه تا مثل بلیط برنده رفتار کنه تا در for بعدی استفاده کنیم و بفهمیم کدوم پردازنده شانس اجرای CPU رو داشته. در مرحله بعدی دوباره روی همه پردازش های runnable پیمایش میکنیم و تمام بلیط های هر کدوم از پردازش ها رو جمع میکنیم و اولین پردازش ای که جمع بلیط هاش از عدد تصادفی عبور بکنه برنده میشه و CPU بهش تخصیص داده میشه .

حالا در مرحله بعدی چون پردازش CPU بهش اختصاص داده شده پس وضعیتش running میشه و با switch() کنترل CPU رو میدیم به این پردازش و پس از اتمام کار این پردازش با CPU دوباره تابع schedule صدا زده میشه تا پردازش بعدی رو مشخص کنه . کد این تابع هم به صورت زیر میشه :

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        intr_on();
        intr_off();

        int total_tickets = 0;

        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE && p->tickets > 0)
                total_tickets += p->tickets;
            release(&p->lock);
        }

        if(total_tickets == 0) {
            asm volatile("wfi");
            continue;
        }

        uint r = rand_int() % total_tickets;
        int cumulative = 0;

        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE && p->tickets > 0) {
```

```

        cumulative += p->tickets;
        if(cumulative > r) {
            p->state = RUNNING;
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            release(&p->lock);
            break;
        }
    }
    release(&p->lock);
}
}
}

```

(4)

حالا باید یک syscall اضافه کنیم اسم این syscall هم sys_settickets و دقیقا کارهایی که در تمرین 1 و 2 برای اضافه کردن syscall انجام میدادم رو انجام دادم و فایل هاش رو ضمیمه کردم. برای اینکار دوتابع مهم نوشتم اولی سمت user و در فایل sysproc.c. در این تابع pid و ticket را میگیریم و سپس تابع رو سمت kernel صدا میزنند که این تابع رو در ادامه توضیح میدم:

```

int
sys_settickets(void)
{
    int pid, tickets;
    argint(0, &pid);
    argint(1, &tickets);
    return ksettickets(pid, tickets);
}

```

تابع مهم دوم رو هم در سمت kernel و در proc.c نوشتم. که همون تابعی هست که توسط user صدا زده شده. در این تابع اگه پرده یافت نشود 1- را ریترن میکنه و یا اگه بلیط نامعتبر بود هم باز 1- ریترن میشود. در غیر این

صورت آگه پردازش یافت شود. اول یک for میزنیم که پردازش رو قبل دستکاری قفل میکنیم تا race اتفاق نیفته و سپس for میزنیم و آگه pid این پردازش برابر pid ورودی باشه میایم تیکت های پردازش رو تغییر میدیم یعنی شانس انتخاب شدن پردازش در زمان بندی عوض میشه. کد این قسمت هم به صورت زیر میشه:

```
int
ksettickets(int pid, int tickets)
{
    struct proc *p;

    if(tickets <= 0)
        return -1;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->tickets = tickets;
            release(&p->lock);
            return 0;
        }
        release(&p->lock);
    }
    return -1;
}
```

(5)

در این فایل LT.c رو نوشتم. در این کد ابتدا چهار پردازش جدید fork میکنم و بهشون بلیط های متفاوت 10و20و40و80 میدم. حالا هر پردازنده با استفاده از while میاد تعداد دفعاته که پردازش CPU رو گرفته میشماره که در آخر این رو در ترمینال چاپ میکنم. در قسمت بعدی کد هر پردازنده میاد pid خودش رو میگیره و براساس اینکه حلقه چندم هستیم میاد بلیط خودش رو انتخاب میکنه و با استفاده از syscall ای که در بالا توضیح دادم (setticket) میاد بلیط هاش رو در kernel تنظیم میکنه. در انتها هم با استفاده از uptime (زمان سیستم بر اساس تیک تایمر) بفهمیم هر پردازش چقدر کار انجام داده. همه پردازش هایی که در یک بازه زمانی میخوان CPU رو

بگیرن و اونی انتخاب میشه که بلیط بیشتری داره که با استفاده از counter معلومش میکنیم . کد این قسمت هم به صورت زیر میشه :

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(void)
{
    printf("LT start\n");

    int tickets[4] = {10, 20, 40, 80};
    int i;

    for(i = 0; i < 4; i++){
        int pid = fork();
        if(pid < 0){
            printf("fork failed\n");
            exit(1);
        }
        if(pid == 0){
            int mypid = getpid();
            int mytickets = tickets[i];
            int r = settickets(mypid, mytickets);
            printf("child pid %d set tickets %d ret %d\n", mypid, mytickets, r);
            if(r < 0){
                exit(1);
            }

            volatile int counter = 0;
            int start = uptime();
            while(uptime() - start < 200){
                counter++;
            }
        }
    }
}
```

```

    printf("pid %d tickets %d counter %d\n", mypid, mytickets, counter);
    exit(0);
}
}

for(i = 0; i < 4; i++){
    wait(0);
}

printf("LT done\n");
exit(0);
}

```

خروجی در انتها به صورت زیر می‌شود:

```

TImarchi-Bagheri@debian:~/s1/xv6-riscv-HW5/xv6-riscv$ CPUS=1 make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nogr
aphic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x
0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ LT
LT start
child pid 4 set tickets 10 ret 0
child pid 6 set tickets 40 ret 0
child pid 5 set tickets 20 ret 0
child pid 7 set tickets 80 ret 0
pid 6 tickets 40 counter 170557
pid 5 tickets 20 counter 68534
pid 7 tickets 80 counter 379836
pid 4 tickets 10 counter 53345
LT done
$ █

```

همان طور که در شکل بالا می‌بینید پردازش‌های که بیشترین بلیط را داشته بیشترین counter رو هم داشته . چون آگه بخوایم بر اساس بلیط از بزرگ به کوچک مرتب کنیم اول pid=7 و سپس pid=6 و سپس pid=5 و سپس pid=4 است و ترتیب counter هم آگه بخوایم از بزرگ به کوچک مرتب کنیم همیشه اول pid=7 و سپس pid=6 و سپس pid=5 و سپس pid=4 که دقیقا مطابق ترتیب بلیط‌ها هست بنابراین کد من درست کار میکند .